Summary and Reflections Report

Prepared for: Supervisor, Grand Strand Systems

Prepared by: Grace Jungclas, MA, AT Ret

Date: August 12, 2026

Summary

The back-end service implementation for Grand Strand Systems required the development and rigorous validation of three core modules: ContactService, TaskService, and AppointmentService. My unit testing approach was engineered to align directly with the software requirements, which dictated strict boundary constraints, non-null guarantees, unique identifier enforcement, and in-memory object manipulation including addition, deletion, and update operations. To align with these requirements, I implemented an equivalence partitioning and boundary value analysis strategy across the testing suite. Each feature was tested using JUnit 5 parameterized and conditional assertions to verify both positive execution paths and negative execution paths. For instance, requirement specifications demanded that a Contact possesses a unique 10-character ID, maximum 10-character first and last names, an exact 10-digit phone number, and a maximum 30-character address. Positive tests verified that valid objects passed into the service methods were successfully instantiated and stored in memory maps, while negative tests ensured that passing null values, out-of-bounds strings, or duplicate unique IDs immediately triggered an IllegalArgumentException (Pressman & Maxim, 2020).

The overall quality and effectiveness of the JUnit test suite were validated using EclEmma code coverage analysis within the Eclipse IDE. High test coverage directly correlates with software reliability by ensuring that all logical branches, conditional checks, and execution

paths inside the service and domain classes have been systematically evaluated (Ammann & Offutt, 2016). Across ContactService.java, TaskService.java, AppointmentService.java, and their respective domain classes (Conact.java, Task.java, Appointment.java), the test suites achieved 100.0% line and branch coverage. Every if/else decision point—including null checks, duplicate key checks via map.containsKey(), and non-existent key lookups—was executed during test runs. By establishing nested test suites using the @Nested annotation for each service operation, the test suite guarantees that no edge cases or unhandled exception paths escape into a production deployment environment.

To ensure that the codebase was technically sound, tests were designed to explicitly assert both state updates and exception propagation rather than relying on simple execution without crashing. Assertions verified state changes directly on objects retrieved from memory. For instance, in the test method testUpdateTaskSuccess(), technical soundness is demonstrated by adding a known object, invoking the service layer's update method, and using assertEquals(expected, actual) to verify that internal object state was accurately mutated in place.

```
125⊖    @Test
126     @DisplayName("Update Task - Success and verified Updated")
127     void testUpdateTaskSuccess() {
128         // add task to service
129         service.addTask(task);
130         // pass taskId from task object and verify it updates name and description without throwing error
131⊖         assertDoesNotThrow( () -> {
132             service.updateTask("1234567891", "Module 3 Assignment", "Introduction into unit testing.");
133         });
134         // verify name and description have been updated
135         assertAll(() -> assertEquals("Module 3 Assignment", task.getName()),
136                 () -> assertEquals("Introduction into unit testing.", task.getDescription()));
137     }
```

Furthermore, exception checks were enclosed in block lambdas to isolate exact fault points during execution, as demonstrated in testContactServiceDuplicateIDThrows():

```java
@Test
@DisplayName("Add Contact - Duplicate ID Throws Exception")
void testContactServiceDuplicateIDThrows() {
    // create another contact with same id as contact initialized in set up
    Contact contact1 = new Contact("1234567891", "John", "Smith", "8007345916", "123 Street Dr");

    // add first contact to Contacts
    service.addContact(contact);
    // check if program will allow duplicative IDs
    assertThrows(IllegalArgumentException.class, () -> service.addContact(contact1));
}
```

Test code efficiency was achieved through test harness optimization using JUnit 5 lifecycle annotations such as @BeforeEach and @Nested. Re-instantiating fresh service instances and default domain objects prior to every individual test method eliminated test pollution and code duplication across test classes (Beck, 2003). Centralizing setup logic ensured that individual test methods avoided redundant instantiation calls, keeping the test file modular, scannable, and maintaining minimal execution overhead:

```java
@BeforeEach
void testTaskServiceSetUp() {
    // clear or re-instantiate objects BEFORE each test run (fresh instances for each test)
    service = new TaskService();
    task = new Task("1234567891", "Module 4 Assignment", "Write unit tests to verify and validate code.");
    }

```

In conclusion, the unit testing strategy executed across all modules successfully validated all core functional requirements while establishing a resilient foundation for back-end service delivery (Pressman & Maxim, 2020). By systematically combining equivalence partitioning, boundary value analysis, and JUnit 5 lifecycle management, the test suite achieved 100.0% line and branch coverage while isolating edge cases and exception handling paths (Ammann & Offutt, 2016). This rigorous approach ensures that the application operates reliably under both nominal and invalid input conditions, verifying that all domain objects and service operations behave as specified without introducing undetected defects into production (Myers et al., 2011).

Reflection:

Several software testing techniques were employed throughout this project, primarily centered around unit-level verification. Equivalence partitioning was used to divide input data

into valid and invalid partitions, while boundary value analysis was applied to test at the extreme edges of input boundaries, such as testing string length limits at 9, 10, and 11 characters. Negative testing and exception testing were used to pass invalid data into methods to ensure the application failed gracefully by throwing an IllegalArgumentException. Finally, state-based testing was utilized to verify object state mutations before and after service executions using JUnit assertions. Conversely, several testing techniques were not used in this scope. Integration testing, which evaluates interactions between separate software modules or external databases, was not employed because objects were stored in an in-memory HashMap. Mocking and stubbing using frameworks like Mockito were unnecessary due to the absence of external dependencies. Performance, load, and stress testing were also excluded as the system was not being evaluated for multi-threaded concurrency or response latency under high traffic. In industry practice, unit testing provides immediate feedback during active development, integration testing verifies database and API boundaries, mocking isolates complex business logic from paid third-party services, and load testing identifies concurrency bottlenecks prior to high-traffic deployment events (Myers et al., 2011).

Adopting the mindset of a software tester requires exercising extreme caution and a healthy skepticism toward developer assumptions. As a tester, one must assume that code will fail, user input will be malformatted, and edge cases will be encountered in production. When testing the service layers, appreciating the interrelationships and complexity of code was essential. The service classes heavily depend on the validation rule established within the underlying domain classes. For example, in ContactService.updateContact(), the service layer fetches a Contact object from its internal map and calls setter methods. Because setter methods in Contact.java perform the actual validation, understanding this interrelation allowed me to design

tests that verify how invalid parameters passed into updateContact() correctly propagate exceptions originating from the domain class setters. Furthermore, to limit reviewer bias—which occurs when a developer tests their own code with implicit assumptions about how it should work—I separated the developer mindset from the tester mindset by designing unit test strictly off the functional requirement specification sheets rather than looking at my implemented Java code. Knowing that IDs must be unique, I deliberately authored tests designed to break the service: attempting to add duplicate IDs sequentially passing null references into methods and supplying boundary-breaking strings.

Maintaining unyielding discipline regarding software quality is a foundational responsibility of a software engineering professional. Cutting corners by skipping unit test creation, ignoring coverage branch highlights, or failing to write negative test cases leads to technical debt, which refers to the implied cost of additional rework caused by choosing an easy short-term solution over a robust design (Fowler, 2019). In production environments, technical debt manifests as fragile codebases, unexpected regression bugs, and catastrophic system failures. To avoid technical debt as a practitioner, I plan to adhere to quality-first principles by integrating unit tests into continuous integration pipelines, performing regular refactoring, and enforcing strict minimum line and branch coverage gates across all back-end service deployments.

In conclusion, adopting a dedicated tester mindset throughout this project highlighted the critical role that rigorous, objective validation plays in modern software engineering (Ammann & Offutt, 2016). By deliberately separating developer assumptions from the testing process and applying structured techniques like boundary value analysis and exception assertion, I was able to systematically uncover edge cases and guard against hidden fault paths (Myers et al., 2011).

Ultimately, maintaining an unyielding commitment to quality and technical discipline is not merely about achieving high coverage metrics, but about preventing technical debt, ensuring system resilience, and delivering dependable software solutions that stand up to real-world demands over time (Fowler, 2019; Pressman &Maxim, 2020).

References

Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.

Beck, K. (2003). *Test-Driven Development: By Example.* Boston: Addison-Wesley Professional. Retrieved from https://learning.oreilly.com/library/view/test-driven-development/0321146530/

Fowler, M. (2019). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Boston: Addison-Wesley Professional. Retrieved from https://learning.oreilly.com/library/view/refactoring-improving-the/9780134757681/

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Hoboken: John Wiley & Sons. Retrieved from https://malenezi.github.io/malenezi/SE401/Books/114-the-art-of-software-testing-3-edition.pdf

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). New York: McGraw-Hill Education. doi:https://doi.org/10.1016/0141-1195(83)90118-3